Lab 4 – Access Control & Network Security

Name: Sicid Sukhra Abdirahman

ID:52215124928

Objective

The objective of this lab is to understand and apply authentication, authorization, multi-factor authentication, network segmentation, firewall rules, and container hardening. The lab also demonstrates least privilege and vulnerability scanning using Docker and Kubernetes.

Environment

The following tools and environment were used to complete this lab:

- Windows 11 with Git Bash for running commands.
- Docker Desktop for containers and network segmentation.
- Nginx for the authentication service.
- Oathtool for TOTP multi-factor authentication.
- kind and kubectl for Kubernetes RBAC.
- Redis and Nginx containers for network segmentation.
- iptables for firewall rules.
- Trivy for vulnerability scanning.

session A – Authentication & Authorization

Task 1 – Authentication: A Password-Protected Service

In this task, I created a password-protected Nginx service using HTTP Basic Authentication. The service was tested without credentials and with valid credentials to verify that only authenticated users could access it.

Commands

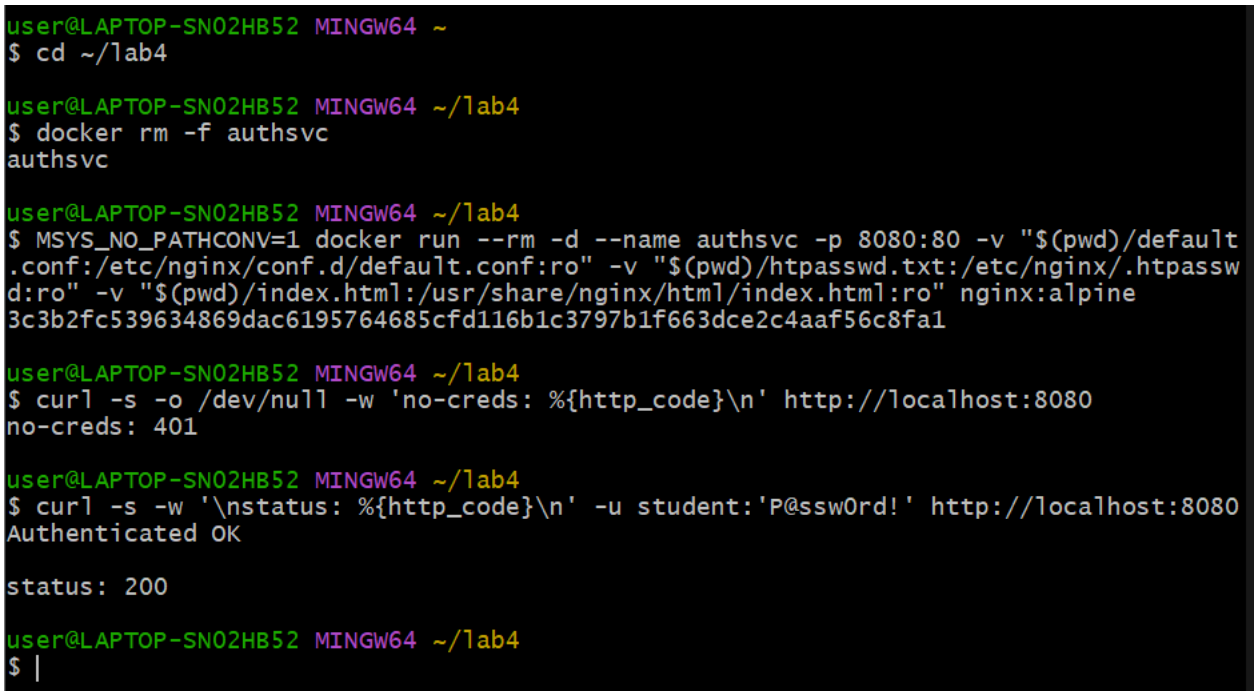
```bash
docker run --rm httpd:alpine htpasswd -nbB student 'P@ssw0rd!' > htpasswd.txt

cat > default.conf <<'EOF'
server {
    listen 80;
    location / {
        auth_basic "Restricted";
        auth_basic_user_file /etc/nginx/.htpasswd;
        return 200 'Authenticated OK\n';
    }
}
EOF

MSYS_NO_PATHCONV=1 docker run --rm -d --name authsvc -p 8080:80 \
  -v "$(pwd)/default.conf:/etc/nginx/conf.d/default.conf:ro" \
  -v "$(pwd)/htpasswd.txt:/etc/nginx/.htpasswd:ro" \
  nginx

curl -s -o /dev/null -w 'no-creds: %{http_code}\n' http://localhost:8080

curl -s -w '\nstatus: %{http_code}\n' \
  -u student:'P@ssw0rd!' http://localhost:8080
```

Result

The request without credentials returned HTTP status `401`, showing that unauthenticated access was rejected.

When the correct username and password were provided, the service returned `Authenticated OK` with HTTP status `200`. This confirmed that HTTP Basic Authentication was working correctly.

Evidence



Task 2 – Add a Second Factor (MFA / TOTP)

In this task, I implemented a second authentication factor using a time-based one-time password (TOTP). Oathtool was used to generate a six-digit code, and the code was validated to demonstrate MFA.

Commands

```bash
docker run --rm ubuntu:24.04 bash -c "apt-get update -qq && apt-get install -y -qq oathtool >/dev/null && oathtool --totp -b '6OFOVZ7C3TOKCNEGGWHPRNTRROWGN4AH'"
```

read -p "Enter the 6-digit code: " CODE

docker run --rm -it ubuntu:24.04 bash -c '

apt-get update -qq &&

apt-get install -y -qq oathtool >/dev/null &&

SECRET="6OFOVZ7C3TOKCNEGGWHPRNTRROWGN4AH";

CODE=$(oathtool --totp -b "$SECRET");

echo "Current 6-digit code: $CODE";

read -p "Enter the 6-digit code: " INPUT;

["$INPUT" = "$CODE"] && echo "MFA OK" || echo "MFA FAILED"

'

```

Result

A six-digit TOTP code was successfully generated and entered for verification. The final output showed:

```text

MFA OK

```

This confirmed that the valid TOTP code was successfully verified as a second authentication factor.

Evidence
```

Task 3 – Authorization: RBAC Roles

In this task, I used Kubernetes Role-Based Access Control (RBAC) to give a developer service account limited permissions. The developer was allowed to read pods but was not allowed to create deployments or delete pods.

Commands

```bash

kind create cluster --name ccse-lab4

kubectl create namespace app

kubectl create serviceaccount dev -n app

kubectl create role dev-role -n app --verb=get,list --resource=pods

kubectl create rolebinding dev-rb -n app \
  --role=dev-role \
  --serviceaccount=app:dev

SA=system:serviceaccount:app:dev
```

kubectl auth can-i list pods -n app --as=$SA

kubectl auth can-i create deploy -n app --as=$SA

kubectl auth can-i delete pods -n app --as=$SA
```

Result

The Kubernetes cluster, namespace, service account, role, and role binding were created successfully.

The authorization tests returned:

```text
yes
no
no
```

The developer service account could list pods but could not create deployments or delete pods. This demonstrated RBAC and the principle of least privilege.

Evidence
```

Session B – Network Security & Hardening

Task 4 – Network Segmentation (Three-Tier)

In this task, I created separate frontend and backend Docker networks. The database was connected only to the backend network, the application was connected to both networks, and the web container was connected only to the frontend network.

Commands

```bash

docker network create frontend-net

docker network create backend-net

docker run -d --name db --network backend-net redis:alpine

docker run -d --name app --network backend-net nginx

docker network connect frontend-net app

docker run -d --name web --network frontend-net nginx
```

```

The network segmentation was tested using:

```bash
docker exec web sh -c 'getent hosts db >/dev/null 2>&1 && echo REACHABLE || echo BLOCKED'

docker exec app sh -c 'getent hosts db >/dev/null 2>&1 && echo REACHABLE || echo BLOCKED'
```

Result

The network tests returned:

```text
BLOCKED
REACHABLE
```

The web container could not directly reach the database, while the application container could reach the database through the backend network.

This demonstrated network segmentation and showed how separating network tiers can reduce lateral movement.

Evidence
```

Task 5 – Firewall Rules (Default-Deny)

In this task, I configured firewall rules using iptables. The default INPUT policy was set to DROP, while TCP port 443 and loopback traffic were explicitly allowed.

Commands

```bash
docker run --rm --cap-add=NET_ADMIN alpine sh -c '\
apk add -q iptables; \
iptables -P INPUT DROP; \
iptables -A INPUT -p tcp --dport 443 -j ACCEPT; \
iptables -A INPUT -i lo -j ACCEPT; \
iptables -L INPUT -n'
```

Result

The firewall output showed:

```text
Chain INPUT (policy DROP)
ACCEPT tcp -- 0.0.0.0/0 0.0.0.0/0 tcp dpt:443
ACCEPT all -- 0.0.0.0/0 0.0.0.0/0
```

The default policy denied incoming traffic unless it was explicitly permitted. TCP port 443 and loopback traffic were allowed.

This demonstrated a default-deny firewall policy and least privilege at the network level.

Evidence

Task 6 – Container / Host Hardening

In this task, I hardened an Nginx container by running it as a non-root user, using a read-only root filesystem, dropping Linux capabilities, preventing new privileges, and using a temporary filesystem for `/tmp`.

The container image was also scanned for HIGH and CRITICAL vulnerabilities using Trivy.

Commands

```bash
MSYS_NO_PATHCONV=1 docker run -d --name hardened \
  --user 1000:1000 \
  --read-only \
  --cap-drop=ALL \
  --security-opt no-new-privileges \
```

```
  --tmpfs /tmp \
  nginxinc/nginx-unprivileged
```

The container was checked using:

```bash
docker ps --filter name=hardened

docker inspect hardened --format 'User={{.Config.User}} ReadOnly={{.HostConfig.ReadonlyRootfs}}'
```

The image was scanned using:

```bash
docker run --rm aquasec/trivy image \
  --severity HIGH,CRITICAL nginx:alpine | head -20
```

Result

The hardened container started successfully.

The inspection showed:

```text
User=1000:1000 ReadOnly=true
```

This confirmed that the container was running as a non-root user and that the root filesystem was read-only.

The Trivy vulnerability scan reported:

```text
Target: nginx:alpine
Total: 2 (HIGH: 2, CRITICAL: 0)
```

The scan identified two HIGH-severity vulnerabilities and no CRITICAL vulnerabilities.

Evidence

Verification

After completing the six tasks, I verified the main security controls implemented in the lab.

The following results confirmed that the configurations worked correctly:

- Basic Authentication returned `401` without credentials and `200` with valid credentials.

- TOTP verification returned `MFA OK`.

- Kubernetes RBAC returned `yes`, `no`, `no`, confirming that the developer service account had only the required permissions.

- Network segmentation returned `BLOCKED` for web-to-database access and `REACHABLE` for application-to-database access.

- The firewall used a default `DROP` policy and allowed TCP port `443`.

- The hardened container was running as `User=1000:1000` with `ReadOnly=true`.

- The vulnerability scan reported `2 HIGH` and `0 CRITICAL` vulnerabilities.

Evidence

```
user@LAPTOP-SN02HB52 MINGW64 ~
$ kubectl get rolebinding dev-rb -n app -o yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  creationTimestamp: "2026-08-27T11:19:25Z"
  name: dev-rb
  namespace: app
  resourceVersion: "799"
  uid: 4d38b5f1-9c08-46f9-b7d3-ce3541c336c9
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: dev-role
subjects:
- kind: ServiceAccount
  name: dev
  namespace: app

user@LAPTOP-SN02HB52 MINGW64 ~
$ docker inspect hardened --format '{{json .HostConfig.CapDrop}}'
["ALL"]

user@LAPTOP-SN02HB52 MINGW64 ~
$
```

Cleanup & Teardown

After completing and verifying the lab, I removed the containers, Docker networks, and Kubernetes cluster created during the exercises.

Commands

```bash
docker rm -f authsvc db app web hardened 2>/dev/null
```

docker network rm frontend-net backend-net 2>/dev/null

kind delete cluster --name ccse-lab4
```

Result

The Docker containers were removed successfully.

The `frontend-net` and `backend-net` networks were removed successfully.

The Kubernetes cluster `ccse-lab4` was also deleted successfully, including its control-plane node.

The cleanup output confirmed:

```text
db
app
web
hardened

frontend-net
backend-net

Deleting cluster "ccse-lab4" ...
Deleted nodes: ["ccse-lab4-control-plane"]
```

Evidence
```

Short-Answer Questions

Q1. Explain the difference between authentication and authorization using Tasks 1 and 3.

Answer:
Authentication checks who the user is. In Task 1, the user had to provide the correct username and password before accessing the service. Authorization checks what an authenticated user is allowed to do. In Task 3, RBAC allowed the dev service account to list pods, but it was not allowed to create deployments or delete pods.

Q2. Why is MFA so effective, and which attacks does it defeat?

Answer:
MFA is effective because it requires more than one way to prove the user's identity. In Task 2, a TOTP code was used as a second factor. Even if an attacker steals the password, they still need the valid TOTP code. This helps protect against attacks such as stolen passwords, password guessing, and credential stuffing.

Q3. How does network segmentation limit the damage of a compromised web server?

Answer:
Network segmentation separates services into different networks and limits direct communication between them. In Task 4, the web container could not reach the database directly, while the app container could reach it. If the web server is compromised, the attacker cannot easily access the database directly. This helps reduce lateral movement and limits the damage of an attack.

Q4. What does a default-deny firewall policy achieve, and how does it relate to cloud security groups?

Answer:
A default-deny firewall blocks traffic unless it is specifically allowed. In Task 5, the INPUT policy was set to DROP, and only required traffic such as port 443 was allowed. Cloud security groups use a similar idea by allowing only the network traffic that a cloud resource needs. This reduces unnecessary access and improves security.

Q5. List the hardening measures you applied and the attack surface each one removes.

Answer:
In Task 6, I applied several container hardening measures. I used a non-root user to reduce the risk of privileged access. I used a read-only filesystem to prevent unwanted file changes. I dropped Linux capabilities to remove unnecessary privileges, and I used no-new-privileges to stop the process from gaining more privileges. I also scanned the image with Trivy to identify known vulnerabilities. These controls reduce the container's attack surface and make it harder for an attacker to misuse the container.

Security Best-Practices Checklist

☑ Service requires authentication (unauthenticated requests rejected).

☑ MFA / second factor implemented and validated.

☑ Authorization enforced by RBAC (least privilege; unauthorised actions denied).

☑ Network segmented so the data tier is unreachable from the front tier.

☑ Default-deny firewall with explicit allow rules.

☑ Container hardened: non-root, minimal, capabilities dropped, read-only; image scanned.